

Big Data Systems – Course Project (Detailed Guide)

Big Data Systems – Course Project (20%)
Semester 2, 1447H (January–May 2026)

1. Purpose and learning outcomes

This project gives you hands-on experience with the full big data analytics pipeline using Apache Spark with Scala. You will:

- Choose a real-world dataset from a public repository and justify your selection.
- Perform thorough data preprocessing (cleaning, integration, reduction, transformation).
- Analyze the data using Spark RDD and SQL operations for discovery and validation.
- Apply machine learning techniques using Spark MLlib/ML to solve a concrete problem or answer a meaningful question.
- Document your methodology and findings in a structured report.
- Present your work clearly in a team presentation.

The project is worth 20% of the course grade and is designed to synthesize concepts from lectures, labs, and assignments into a real-world-style deliverable.

By completing this project, you should be able to:

1. Select and evaluate datasets based on suitability, size, quality, and alignment with learning goals.
 2. Design and implement data preprocessing pipelines that handle cleaning, integration, reduction, and transformation at scale.
 3. Use Apache Spark RDD and DataFrame APIs to perform exploratory analysis and compute meaningful statistics.
 4. Write scalable Spark SQL queries that answer domain-specific questions efficiently.
 5. Apply machine learning with Spark MLlib/ML, including feature engineering, model training, evaluation, and interpretation.
 6. Document and communicate technical work clearly to non-technical and technical audiences.
-

2. Teams, roles, and expectations

- Work in a group of four students. Teams of fewer than four require explicit instructor approval.
- All team members are expected to contribute equally to every phase (data selection, preprocessing, RDD, SQL, ML, report, and presentation).
- One member may "lead" a phase (e.g., ML lead), but every member must understand the dataset, pipeline, and results and be able to explain them.

Teamwork guidance

- Use a shared repository (GitHub or similar) or shared folder for version control when possible.
 - Set internal deadlines earlier than official ones to allow for review and revision.
 - Establish clear role assignments for each phase so work is distributed fairly.
 - Communicate regularly: weekly check-ins help catch problems early.
 - If a member is not contributing, inform the instructor before Week 12; grades may differ between members.
-

3. Project timeline (phases and deadlines)

- Phase 1 – Data selection: Sunday, February 15, 2026 (Week 4) – 3%
 - Phase 2 – Data preprocessing: Sunday, March 1, 2026 (Week 6) – 4%
 - Phase 3 – RDD operations: Sunday, March 15, 2026 (Week 8) – 3%
 - Phase 4 – SQL operations: Sunday, March 29, 2026 (Week 10) – 3%
 - Phase 5 – Machine learning: Sunday, April 19, 2026 (Week 13) – 4%
 - Phase 6 – Final report & presentation: Sunday, May 10, 2026 (Week 16) – 3%
-

4. Phase 1 – Data selection and problem definition (3%)

4.1 Where to find data

Use public, ethically safe datasets from sources such as:

- UCI Machine Learning Repository:
 - <http://archive.ics.uci.edu/ml/>
- Kaggle Datasets:
 - <https://www.kaggle.com/datasets>
- KDnuggets Datasets:
 - <https://www.kdnuggets.com/datasets/index.html>
- Data.World:
 - <https://data.world>
- World Data Atlas (Knoema):

- <https://knoema.com/atlas>
- Government open data portals (e.g., city or national open data sites)

If you want to use another source, verify that the data is publicly shareable and suitable for educational use.

4.2 Dataset requirements

Minimum (hard requirement):

- At least 50,000 rows or multiple related tables/files such that the project is non-trivial (cannot be fully explored in a spreadsheet).

Preferred (for strongest projects):

- $\geq 500,000$ rows or ≈ 1 GB total, or a multi-file/multi-year dataset that clearly benefits from Spark.

Other requirements:

- Variety: A mix of numeric, categorical, temporal, and/or text attributes.
- Documentation: Data schema or sufficient description of attributes and units.
- Quality: Some missing values or noise (so preprocessing is meaningful) but not so messy that cleaning dominates the whole project.
- Ethics: No confidential or sensitive information; data must be anonymized and clearly allowed for educational use.

4.3 Problem statement

Define a clear analytical or predictive problem. It should answer:

- What are you trying to analyze or predict?
- Why is this interesting or important?
- Who might use these insights?

Examples:

- "Predict whether a bank customer will subscribe to a term deposit."
- "Analyze factors contributing to traffic congestion in a city."
- "Recommend movies to users based on past ratings."

Avoid extremely vague goals like "understand the data" without a specific question.

4.4 Phase 1 deliverable

Submit a short proposal (max 2 pages) containing:

- Group members: Names, IDs, and email addresses of all 4 team members.
- Dataset name, domain, and source URL.
- Approximate dataset size: rows, file size, number of files/tables.
- Main attributes and their types: numeric, categorical, temporal, text.
- Problem statement and motivation.

- Brief explanation: Why this dataset is suitable for Spark and this project.

4.5 Evaluation (Phase 1 – 3%)

- Excellent (3%) – Dataset clearly suitable and non-trivial; problem is well-defined, interesting, and achievable; justification is strong.
 - Good (2%) – Dataset acceptable but small/simple or weakly motivated; problem somewhat vague.
 - Satisfactory (1%) – Dataset borderline small or problem unclear; proposal needs minor revision.
 - Poor (0%) – Dataset too small or inappropriate; objectives unclear; or approval not obtained.
-

5. Phase 2 – Data preprocessing (4%)

Goal: Transform raw data into a clean, consistent, analysis-ready dataset.

Your preprocessing must include all four components: cleaning, integration, reduction, transformation.

5.1 Data cleaning

Handle basic data quality issues:

- Missing values: Decide case-by-case whether to drop, impute, or encode them as a separate category.
- Errors and inconsistencies: Remove or correct impossible values (e.g., negative ages, invalid dates) and fix inconsistent codes.
- Duplicates: Detect and remove duplicate records when appropriate.
- Outliers: Identify extreme values and decide whether to remove, cap, or keep them, with justification.
- Standardization: Ensure consistent formats for dates, currencies, and units.

Report simple before/after statistics (e.g., how many rows were removed, how many missing values were handled).

5.2 Data integration (if applicable)

If your project uses multiple files or tables:

- Join them on appropriate keys (e.g., customer ID, trip ID, timestamp + location).
- Resolve conflicts when the same attribute appears in multiple sources.
- Document the final integrated schema (attribute name, type, description).

If your project uses a single file, still document the schema clearly.

5.3 Data reduction

Reduce data volume or dimensionality while keeping it useful:

- Sampling: Random or stratified sampling to manage size.
- Aggregation: Summarizing detailed events into daily/weekly totals or per-entity metrics.
- Feature selection: Dropping irrelevant or redundant features.

For each reduction step, explain:

- Why it was necessary.
- How much data was removed or aggregated.
- Possible impact on analysis or model.

5.4 Data transformation

Prepare data for analysis and ML:

- Convert data types appropriately (e.g., strings to numeric, timestamps to date/hour).
- Encode categorical variables (e.g., one-hot encoding or index-based encoding for MLlib).
- Normalize or scale numerical features when needed.
- Engineer domain-relevant features (e.g., "is_weekend", "session_length", "avg_spend_per_visit").

Document each step and its rationale briefly.

5.5 Phase 2 deliverables

- Scala/Spark code implementing all preprocessing steps.
- A snapshot (10–20 rows) of the final preprocessed dataset.
- A 1–2 page summary describing:
 - Main data quality issues.
 - Cleaning/integration/reduction/transformation steps with key statistics.
 - Justification for major decisions.

5.6 Evaluation (Phase 2 – 4%)

- Excellent (4%) – All four components applied thoughtfully; well-documented with code and before/after statistics; decisions clearly justified.
- Good (3%) – Most steps implemented; minor gaps in documentation or reasoning.
- Satisfactory (2%) – Minimal cleaning or transformation; important steps present but weakly explained.
- Weak (1%) – Minimal preprocessing; important steps missing.
- None (0%) – Raw or nearly raw data; preprocessing missing or non-functional.

6. Phase 3 – RDD operations in Spark (3%)

Goal: Demonstrate effective use of Spark RDDs for exploration and transformation.

6.1 Requirements

- Implement at least five distinct RDD transformations, such as:
 - map, filter, flatMap, reduceByKey, groupByKey, join, distinct, sortByKey.
- Implement at least five distinct RDD actions, such as:
 - count, take, collect (on small RDDs), reduce, saveAsTextFile, first, foreach.

Each operation must be non-trivial and relevant to your dataset and problem, not just to satisfy the count.

6.2 Example RDD tasks

Depending on your domain, you might:

- Compute per-customer statistics (e.g., number of transactions, total and average spending).
- Find top N items/routes/locations by frequency or revenue.
- Compute distributions (e.g., delay minutes grouped by airline, rating grouped by genre).
- Derive intermediate features (e.g., session length, number of pageviews per session).

6.3 Phase 3 deliverables

For each transformation/action:

- Provide a Scala code snippet.
- Show a sample of the output (not huge dumps).
- Add 1–2 sentences explaining what the operation reveals or how it transforms the data.

Deliver:

- Scala/Spark code file with all RDD operations, clearly commented.
- A short 1–2 page analysis summarizing main insights from RDD work and how they support your problem.

6.4 Evaluation (Phase 3 – 3%)

- Excellent (3%) – ≥ 5 distinct, meaningful transformations and ≥ 5 actions; code correct and clear; each operation has sample output and interpretation.

- Good (2%) – Requirements met but some operations are trivial or weakly explained.
 - Satisfactory (1%) – Fewer than 5 operations; some repetitive patterns; weak explanation.
 - Weak (0.5%) – Barely meets minimum; many trivial operations; minimal analysis.
 - None (0%) – RDD work missing, non-functional, or far below requirements.
-

7. Phase 4 – SQL operations in Spark (3%)

Goal: Use Spark SQL and DataFrames to answer structured analytical questions.

7.1 Requirements

- Convert your preprocessed dataset to a DataFrame.
- Register it as a temporary view or table.
- Write at least five non-trivial SQL queries, including a mix of:
 - Aggregations (GROUP BY, HAVING).
 - Filtering and ranking (WHERE, ORDER BY, LIMIT, window functions).
 - Joins between tables (if using multiple datasets).
 - Subqueries or common table expressions (CTEs).
 - Statistical summaries (distinct counts, percentiles, variance).

Each query must answer a concrete question related to your project.

7.2 Example SQL tasks

- "What is the average transaction amount by customer segment and country?"
- "Which routes have the highest average delay?"
- "Top 10 products by revenue in each category."
- "Monthly trend of new users or events."

7.3 Phase 4 deliverables

- Scala/Spark code file containing DataFrame creation and all SQL queries.
- SQL statements and sample outputs for each query.
- A 1–2 page explanation describing what each query reveals and how it connects to your problem.

7.4 Evaluation (Phase 4 – 3%)

- Excellent (3%) – ≥ 5 non-trivial, correct queries; queries clearly support project goals and show understanding of grouping, joining, and filtering; outputs interpreted.

- Good (2%) – Queries mostly correct but some simple or weakly tied to objectives; limited interpretation.
 - Satisfactory (1%) – Queries present but mostly simple; weak connection to project goals.
 - Weak (0.5%) – Very few queries; significant SQL errors.
 - None (0%) – No meaningful SQL work.
-

8. Phase 5 – Machine learning with Spark (4%)

Goal: Apply machine learning to solve your defined problem and interpret the results.

8.1 Typical ML pipeline

1. Feature engineering and preparation

- Use VectorAssembler to combine features into a single feature vector.
- Encode categorical variables with StringIndexer, OneHotEncoder, or similar.
- Scale/normalize features if needed (e.g., StandardScaler, MinMaxScaler).

2. Model training

- Choose at least one meaningful model aligned with your problem type:
 - Classification: Logistic Regression, Decision Tree, Random Forest, Gradient Boosted Trees, etc.
 - Regression: Linear Regression, Decision Tree Regressor, Random Forest Regressor, etc.
 - Clustering: K-Means, Gaussian Mixture Models.
- Split data into training and test sets (e.g., 70/30) using a fixed random seed.

3. Model evaluation

- Compute appropriate metrics:
 - Classification: Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrix.
 - Regression: RMSE, MAE, R^2 .
 - Clustering: Silhouette score or similar.
- Compare your model to a simple baseline (e.g., majority class for classification, mean prediction for regression).

4. Interpretation

- Analyze feature importance or model coefficients when available.
- Explain what the model has learned in domain terms.
- Discuss where the model performs well or poorly, and possible sources of bias or error.

8.2 Key requirements and common pitfalls

- Use a clear train/test split with a fixed seed for reproducibility.
- Avoid data leakage: do not fit preprocessing steps on the full dataset and then split; fit on training data only, then apply to test data.
- Even if performance is modest, focus on correct methodology and honest interpretation rather than chasing high accuracy.

8.3 Phase 5 deliverables

- Scala/Spark code including feature preparation, model training, and evaluation.
- Training and test metrics for your model(s).
- Feature importances or coefficients (if supported).
- A 2–3 page analysis covering:
 - Problem formulation (target variable, input features).
 - Choice of model and why it fits the task.
 - Evaluation metrics and what they mean.
 - Interpretation of key features and limitations of the model.

8.4 Evaluation (Phase 5 – 4%)

- Excellent (4%) – Proper pipeline with train/test split; suitable model; appropriate metrics; clear interpretation; discussion of strengths/limitations.
 - Good (3%) – Model built and evaluated, but methodology or explanation is incomplete.
 - Satisfactory (2%) – Basic ML work; some methodological issues; limited interpretation.
 - Weak (1%) – Very basic or partially incorrect ML; unclear split; minimal evaluation.
 - None (0%) – No real ML component; non-functional code; or meaningless model.
-

9. Phase 6 – Final report and presentation (3%)

9.1 Final report (10–15 pages)

Recommended structure:

1. Introduction (1–2 pages)
Background, problem statement, motivation, objectives, and scope.
2. Data selection and understanding (1–2 pages)
Dataset source, domain, size, attributes, and initial observations.
3. Data preprocessing (2–3 pages)
Issues identified, detailed cleaning/integration/reduction/transformation steps, before/after statistics, justifications.

4. RDD operations (2–3 pages)
Main RDD operations used, code snippets, sample outputs, and interpretations.
5. SQL operations (1–2 pages)
Key queries, their purpose, sample results, and insights.
6. Machine learning operations (2–3 pages)
ML problem formulation, features, model(s) used, metrics, interpretation, and discussion of limitations.
7. Conclusion and future work ($\approx$ 1 page)
Summary of main findings, how well the model addressed the problem, challenges, and potential extensions.

Use figures, tables, and bullet points to improve readability rather than long blocks of text.

9.2 Presentation ($\approx$ 10 minutes)

Suggested outline:

- Problem & dataset (2 minutes) – What you studied and why it matters.
- Preprocessing & exploration (2 minutes) – Main challenges and interesting patterns.
- RDD & SQL highlights (2 minutes) – Example insights from computations and queries.
- Machine learning (3 minutes) – Model choice, metrics, and what the model learned.
- Q&A (1 minute) – Answer questions from classmates or instructor.

Requirements:

- All four team members should speak for roughly equal time.
- Slides should emphasize clarity: limited text, readable figures and tables, and key code snippets where necessary.

9.3 Phase 6 deliverables

- Final report (PDF).
- Presentation slides (PDF or PPT).
- All Scala/Spark source code organized in folders.
- Processed dataset (or precise instructions to regenerate it from raw data).
- A README.txt explaining:
 - How to set up the environment.
 - How to run each script or notebook.
 - Spark version and any special dependencies.

9.4 Evaluation (Phase 6 – 3%)

- Excellent (3%) – Report follows structure, is well-organized, clear, and mostly error-free; includes supporting figures/tables/code snippets; presentation is on time, within 10 minutes, visually clear, and shared fairly between all 4 members.
 - Good (2%) – Report mostly complete; presentation mostly well-delivered; minor imbalances in speaking roles.
 - Satisfactory (1%) – Some missing sections or weak explanations; visuals or timing issues; notable imbalances in presentation.
 - Weak (0.5%) – Report incomplete; presentation weak or unbalanced.
 - None (0%) – Report missing major parts; presentation not delivered or extremely poor.
-

10. Grading rubric overview (20%)

- Data selection & understanding – 3%
- Data preprocessing – 4%
- RDD operations – 3%
- SQL operations – 3%
- ML operations – 4%
- Report & presentation – 3%

Total: $3 + 4 + 3 + 3 + 4 + 3 = 20\%$

11. Submission format and checklist

11.1 Folder structure

Submit a compressed folder named:

GroupName_BigDataProject.zip

with the following structure:

text

GroupName_BigDataProject/

├── README.txt

├── FinalReport.pdf

├── Presentation_slides.pdf

├── code/

| └── 01_DataPreprocessing.scala

```
| | — 02_RDDOperations.scala
| | — 03_SQLOperations.scala
| | — 04_MachineLearning.scala
| | — utility_functions.scala
| — data/
| | — raw_dataset.csv (or link/instructions)
| | — preprocessed_dataset.parquet (or CSV)
| — results/
| | — rdd_output.txt
| | — sql_results.csv
| | — ml_metrics.txt
```

11.2 Checklist before submitting

- All 4 team members' names and IDs in proposal.
 - Dataset approval obtained (Phase 1).
 - Preprocessing pipeline implemented and summarized (Phase 2).
 - RDD operations coded, tested, and interpreted (Phase 3).
 - SQL queries implemented, tested, and interpreted (Phase 4).
 - ML model trained, evaluated, and analyzed (Phase 5).
 - Final report completed (10–15 pages).
 - Slides prepared for ~10-minute presentation.
 - All code organized and commented.
 - Processed data and/or regeneration instructions included.
 - README.txt explains setup and execution.
-

12. Policies and support

- Academic integrity: You may consult books, online documentation, and example code, but copying full solutions is prohibited. Cite any significant external resources.
- Group issues: Report non-contributing members before Week 12; the instructor may assign different grades.
- Team management: With 4 members, assign clear roles per phase (data lead, preprocessing lead, ML lead, reporting lead) to distribute work fairly.
- Questions: Use office hours, lab time, or email to ask for help—do not wait until the last week.